

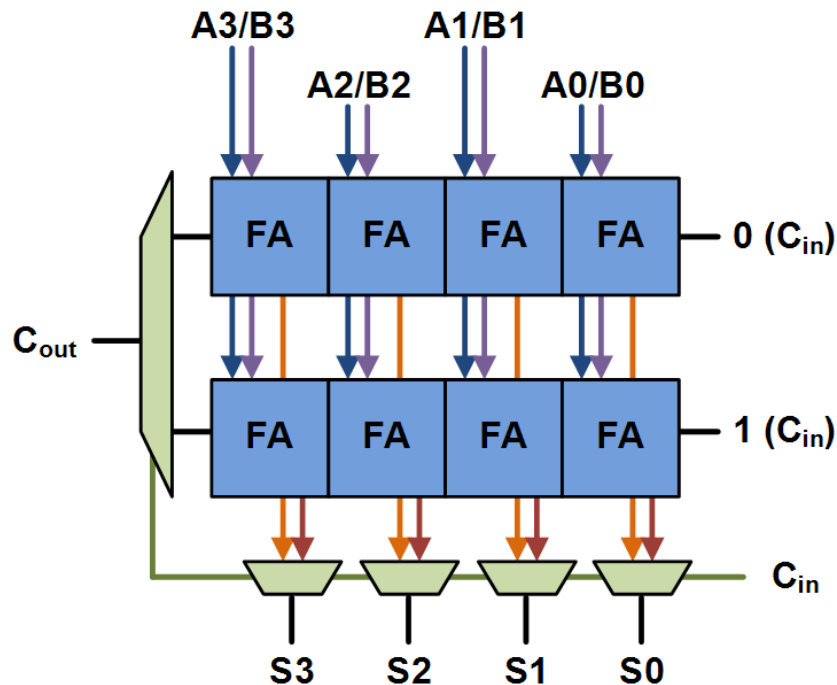
16-bit Multicycle CPU Design

Arisa Arzan Pour — 402105608

Modules

1. Carry Save Adder (CSA) Module

In this module we built an adder using the carry-save adder (CSA) method. The figure below shows the schematic view of this adder, which was used as the basis for the code:



Schematic of the 4-bit carry-save adder (CSA) block.

The **csa** module was written using gate-level modeling to implement the structure shown above. Then, in the **bit16_csa** module, we instantiated the CSA module four times, passing the carry-out of each block into the next block so that carry propagation occurs across the full width. Each of these four instances produces 4 bits of the result, and the final instance also gives us the overall carry-out (C_{out}). A **Mux** module was also written to implement simple multiplexer logic.

2. Multiplication Module (Karatsuba Method)

This part was implemented according to the description given in the assignment document. We start by describing the **shiftadd** module.

This module takes two 8-bit numbers as input and outputs a 16-bit result containing the product of the two inputs. It also outputs a **done** signal, which is set to 1 once the operation is finished; this signal is later used by the higher-level algorithm.

We use a 4-state FSM to carry out this method. First, we check whether the reset signal is 1; if so, all signals are reset and initialized to 0.

Next, in the first FSM state, if the **start** signal (a module input marking when the multiplication should begin) is 1, we move to the next state. In state **01**, we prepare the operands for multiplication:

```
2'b01: begin
    result <= 16'b0;
    multiplicand <= {8'b0, A};
    multiplier <= B;
    count <= 3'd7;
    state <= 2'b10;
end
```

For **multiplicand**, input A is zero-extended to 16 bits so it can be used in the shifting stage that follows.

We then move to state **10**. In this state, if the least-significant bit of **multiplier** is 1, the current value of **multiplicand** is added into **result** (the initial value of **result** is zero). Afterward — regardless of whether that bit was 1 or 0 — **multiplicand** is shifted one bit to the left and **multiplier** is shifted one bit to the right, so the next digit can be examined.

This is exactly equivalent to the long multiplication we do by hand, except that since the digits are only 0 and 1, no actual internal multiplication is required. At this stage we check whether this cycle has been repeated 8 times. Once it has, we move to the final FSM state, store the final result in the corresponding variable, set the **done** signal to 1, and return to state 00.

Now we turn to the main **multiply** module. This module receives signed inputs and determines their signs from their MSBs. Using XOR, we determine what sign the product should have, so it can eventually be applied to the result. After determining the sign of the inputs, if either is negative we compute its two's complement so that the multiplication itself is carried out on positive numbers.

Following the formulas given in the assignment document, the numbers are split into parts, and by instantiating the **shiftadd** module three times, the required partial products are computed.

An FSM is then defined for this module as well:

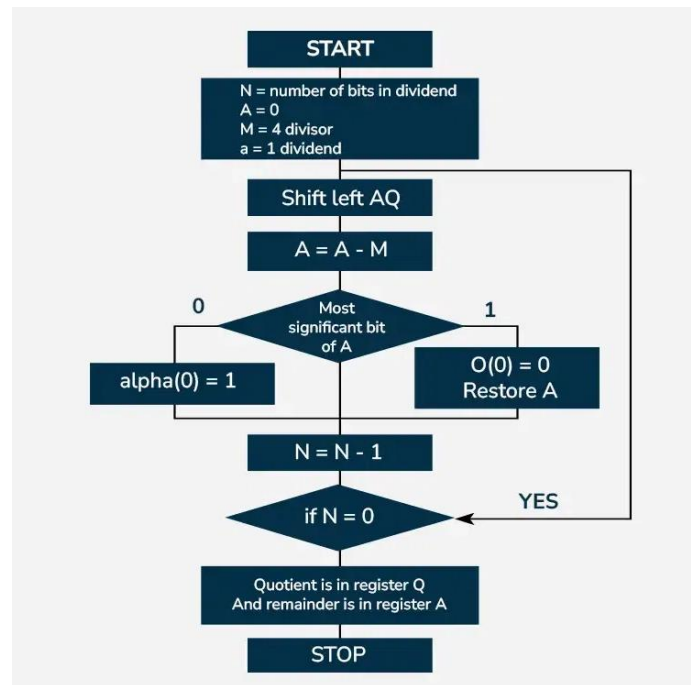
```
parameter WAIT = 2'b00, START_MULT = 2'b01, CALCULATE = 2'b10, DONE = 2'b11;
```

As in the previous module, we first check the reset signal. If it is not asserted, we move to the first state of the FSM. In this state, once the **start** signal (an input marking the beginning of the process) is received, the start signals of the three instances are set to 1 and we move to the next state. In that next state, after those start signals have been asserted, they must be de-asserted, and then we move on to the next state.

In the following (calculation) state, the final product is computed according to the formulas from the assignment document, once the done signals of all three instances — responsible for the three 8-bit multiplications — have been asserted, so that we have the complete and correct partial products to work with. In this same state, based on the sign determined earlier, the final signed result is placed into the appropriate variable, and finally we move to the terminal **done** state, where the module's **done** output is set to 1 so other modules know the operation has finished, and we return to the first state.

3. Division Module (Restoring Method)

This module is structurally similar overall to the previous module, with the difference that it follows the algorithm shown below:



Restoring division algorithm flowchart.

Variable names were chosen to match this flowchart exactly. Finally, this module also has a **done** signal, which becomes 1 in the FSM's final state so that the higher-level modules know the operation has completed.

4. ALU Module

The ALU module contains all of the modules described above. It receives a clock signal, a reset signal, two signed 16-bit numbers, and an opcode; it feeds these values and data into the instantiated modules above and outputs the result according to the given opcode. It also outputs a **done** signal indicating that the ALU's work is complete.

In the FSM portion of this module there are three states: **IDLE**, **EXECUTE**, **DONE**. In the **IDLE** state, the control signals are first cleared to 0; if the **start** signal is 1, a **case** statement on the opcode identifies the intended R-type instruction, and the control signals and inputs of the corresponding sub-modules are set accordingly. If the incoming opcode is not among the valid opcodes, the ALU output will be the expression **dead**.

Finally, all valid cases in this state move on to the next state, **EXECUTE**:

```
EXECUTE: begin
  case (opcode)
    3'b000, 3'b001: begin
      // for add and sub we take the result immediately (no sequential
      // element, all gates)
      result <= add_result;
```

```

        done <= 1;
        state <= DONE_STATE;
    end
    3'b010: begin
        // for Mul and Div we need to wait until we receive the done signal
        // from their modules, then we can proceed to DONE state and set
        // the done signal.
        if (mul_done) begin
            result <= mul_out[15:0];
            done <= 1;
            state <= DONE_STATE;
        end
    end
    3'b011: begin
        if (div_done) begin
            result <= div_quotient;
            done <= 1;
            state <= DONE_STATE;
        end
    end
endcase
end

```

After this state we move to the final state, in which we simply return to the initial **IDLE** state.

5. Memory Module

The code below corresponds to the memory module. Key explanations are given as comments.

```

module memory (
    input clk,
    input write_enable,
    input [15:0] address,
    input signed [15:0] write_data,
    output signed [15:0] read_data
);
    reg signed [15:0] mem [0:2047]; // memory size reduced so that
                                    // synthesis is possible

    always @(posedge clk) begin
        if (write_enable) begin // synchronous write when write_enable = 1
            mem[address] <= write_data;
        end
    end

    assign read_data = mem[address]; // asynchronous read
endmodule

```

6. Register File Module

The code below corresponds to the register file module. Key explanations are given as comments.

```
module register_file (
    input wire clk,
    input wire reset,
    input wire write_enable,
    input wire [1:0] read_addr1,
    input wire [1:0] read_addr2,
    input wire [1:0] write_addr,
    input wire [15:0] write_data,
    output reg [15:0] read_data1,
    output reg [15:0] read_data2
);
    reg [15:0] X[0:3]; // the registers in use

    always @(negedge clk) begin
        // reading is performed on the negative edge of the clock
        read_data1 <= X[read_addr1];
        read_data2 <= X[read_addr2];
    end

    always @(posedge clk or posedge reset) begin
        // writing is performed on the positive edge of the clock;
        // if reset = 1, all registers are cleared to 0
        if (reset) begin
            X[0] <= 16'b0;
            X[1] <= 16'b0;
            X[2] <= 16'b0;
            X[3] <= 16'b0;
        end else if (write_enable) begin
            X[write_addr] <= write_data;
        end
    end
endmodule
```

7. Control Unit Module

Now we look at the most important module. In this module, the outputs of all the modules discussed so far are taken as inputs, and conversely, our outputs serve as the inputs of those modules, so that the logic governing their use can be implemented.

We also have a **ready** output signal, which becomes 1 whenever an instruction has finished executing.

The main structure of this module is a five-state FSM, similar to MIPS, consisting of **IF**, **ID**, **EXE**, **MEM**, and **WB**.

In state **IF**, the PC is first supplied as the memory address so that the instruction can be fetched from memory and executed in the following stages (PC is an internal register variable; it is set to 0 during reset, and is incremented in the final state, then moves forward). The remaining signals stay at 0. We then move to state **ID**.

In this stage, the data read from memory is loaded into a register called **instruction** so that it can be decoded. First the opcode is extracted, and based on it — depending on whether the instruction is R-type or M-type — the remaining bits are interpreted accordingly. After this stage, the **EXE** state executes.

In this state, for R-type opcodes, the ALU inputs are supplied to the ALU; these numeric values come from the register file outputs, whose register addresses were supplied via **address_read**, giving us the register contents that we use in the ALU. We then assert the ALU's start signal, and once the ALU's **done** signal becomes 1, we clear its start signal and move to **WB**, since these instructions do not need the **MEM** stage.

For M-type instructions, similarly to R-type, we supply the ALU inputs it needs, and once its **done** signal becomes 1, we take the ALU output as the address to be given to memory (to write to or read from). The remaining parts follow the same description as the R-type case. These instructions then move to the **MEM** state.

In state **MEM**, depending on whether the instruction is Load or Store, the memory control signals are asserted. For a Load instruction, after this stage we move to **WB**; for a Store instruction, we return to **IF** and also update the PC.

Finally, in the last state, **WB**, we assert the register-file control signals for writing. For Load instructions we store the memory output, and for R-type instructions we store the ALU output.

8. CPU_top Module

This module has two inputs: clock and reset. No logic is implemented in this module itself; we simply declare all the signals used in the previous modules as wires and instantiate the modules, creating the connections between them.

Testbench Outputs

1. Memory, Register, and Instruction Values in the First Testbench

```
uut.memory_inst.mem[0] = 16'h8205; // LOAD R0, R1, #5
uut.memory_inst.mem[1] = 16'h0300; // ADD R0, R1, R2
uut.memory_inst.mem[2] = 16'hA206; // STORE R0, R1, #6
uut.memory_inst.mem[3] = 16'h3B00; // SUB R3, R1, R2
uut.memory_inst.mem[4] = 16'h5080; // MUL R2, R0, R1
uut.memory_inst.mem[5] = 16'h6D80; // DIV R1, R2, R3

uut.regfile.X[0] = 16'd0;
uut.regfile.X[1] = 16'd10;
uut.regfile.X[2] = 16'd20;

uut.memory_inst.mem[15] = 16'd123;
```

Observed output:

```
VSIM 190> run -all
# Time: 0 | R0: 0 | R1: 0 | R2: 0 | R3: 0 | Mem[16]: x
# Time: 20 | R0: 0 | R1: 10 | R2: 20 | R3: 0 | Mem[16]: x
# Time: 105 | R0: 123 | R1: 10 | R2: 20 | R3: 0 | Mem[16]: x
# Time: 175 | R0: 30 | R1: 10 | R2: 20 | R3: 0 | Mem[16]: x
# Time: 245 | R0: 30 | R1: 10 | R2: 20 | R3: 0 | Mem[16]: 30
# Time: 315 | R0: 30 | R1: 10 | R2: 20 | R3: -10 | Mem[16]: 30
# Time: 525 | R0: 30 | R1: 10 | R2: 300 | R3: -10 | Mem[16]: 30
# Time: 775 | R0: 30 | R1: -30 | R2: 300 | R3: -10 | Mem[16]: 30
#
# --- Final Register Values ---
# R0 = 30
# R1 = -30
# R2 = 300
# R3 = -10
# Mem[16] = 30
# ** Note: $stop : C:/Users/AA_NOIR/Desktop/project_dsd/tb_1.v(60)
```

2. Memory, Register, and Instruction Values in the Second Testbench

```
uut.memory_inst.mem[0] = 16'h2300; // SUB R0, R1, R2 => R0 = R1 - R2 = -17 - (-5) = -12
uut.memory_inst.mem[1] = 16'h5900; // MUL R3, R0, R2 => R3 = -12 * -5 = 60
uut.memory_inst.mem[2] = 16'h7680; // DIV R2, R3, R1 => R2 = 60 / -17 = -3 (integer div)
uut.memory_inst.mem[3] = 16'h0900; // ADD R1, R0, R2 => R1 = -12 + (-3) = -15
uut.memory_inst.mem[4] = 16'hB7FF; // STORE R2, R3, #-1

uut.regfile.X[0] = 16'd0;
uut.regfile.X[1] = -17;
uut.regfile.X[2] = -5;
uut.regfile.X[3] = 0;
```

Observed output:

```
-----
# Time: 0 | R0: 0 | R1: 0 | R2: 0 | R3: 0 | Mem[59]: x
# Time: 20 | R0: 0 | R1: -17 | R2: -5 | R3: 0 | Mem[59]: x
# Time: 95 | R0: -12 | R1: -17 | R2: -5 | R3: 0 | Mem[59]: x
# Time: 305 | R0: -12 | R1: -17 | R2: -5 | R3: 60 | Mem[59]: x
# Time: 555 | R0: -12 | R1: -17 | R2: -3 | R3: 60 | Mem[59]: x
# Time: 625 | R0: -12 | R1: -15 | R2: -3 | R3: 60 | Mem[59]: x
# Time: 695 | R0: -12 | R1: -15 | R2: -3 | R3: 60 | Mem[59]: -3
#
# --- Final Register Values ---
# R0 = -12
# R1 = -15
# R2 = -3
# R3 = 60
# Mem[59] = -3
```